

Secure Exit

SIMATS
ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

KALAHSTHI RAMYA
192311149RA

CO3-AT1-Assignment

[← Back](#)

QUESTIONS

Q1

Banking Transaction System –
Exception Handling
10 marks

Q2

Hospital Patient Registration –
User-Defined Exception
10 marks

Q3

E-Commerce Order Processing
– Built-in Exceptions
10 marks

Q4

Online Food Delivery – Multiple
Threads and Execution Order
10 marks

Q5

Manufacturing Plant –
Producer-Consumer Problem
10 marks

5/5 answered

Q4

**Online Food Delivery – Multiple Threads
and Execution Order**10
marks

An online food-delivery application uses three threads: Order Processing, Payment Processing, and Delivery Assignment.

Develop a Java program in which these activities are represented by separate threads. Analyze the problem if all three threads are started independently.

Modify the program so that:

- Order processing completes before payment processing.
- Payment processing completes before delivery assignment.
- The program uses appropriate Java thread-management techniques.
- The output clearly shows the required execution order.

Explain the difference between simply starting all threads and explicitly controlling their execution order.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Order=Pizza; Payment=UPI; Delivery=Agent A Order →
Payment → Delivery Normal execution

Order=Burger; Payment=Card; Delivery=Agent B Order

→ Payment → Delivery Concurrency / design

Payment=Failed Delivery assignment should not
proceed Exception / boundary

Your Code

```

1 import java.util.*;
2
3 public class Main{
4     public static void main(String[] args){
5         Scanner sc = new Scanner(System.in);
6
7         try{
8             String foodItem = sc.nextLine().
9             String paymentMethod = sc.nextLi
10
11             boolean isPaymentSuccessful = sc
12             sc.nextLine();
13
14             String agentName = sc.nextLine()
15
16             runDeliveryWorkflow(foodItem, pay
17         }catch(Exception e){
18             System.out.println("Invalid input
19         }finally{
20             sc.close();
21         }
22     }
23
24     public static void runDeliveryWorkflow(S
25         Thread orderThread = new Thread(() -
26         System.out.println("[Order Threa
27     });
28
29     Thread paymentThread = new Thread(()
30         System.out.println("[Payment Thr
31     });
32     Thread deliveryThread = new Thread((
33         System.out.println("[Delivery Th
34     });
35
36     try{
37         orderThread.start();
38         orderThread.join();

```

```
39
40         paymentThread.start();
41         paymentThread.join();
42
43         if(!isPaymentSuccessful){
44             System.out.println("[System
45             return;
46         }
47
48         deliveryThread.start();
49         deliveryThread.join();
50
51         System.out.println("\n--- Workfl
52     } catch (InterruptedException e) {
53         System.out.println("System execu
54         Thread.currentThread().interrupt
55     }
56 }
57 }
```



Input

Burger
Card
B



Output

Invalid input format. Please restart and try again.

✓ Submitted